PROJECT REPORT

Huff

A Huffman File Compressor

Lossless compression with a C++17 command-line tool and a browser interface sharing the HUF1 archive format

Project area	Data Structures and Algorithms
Implementation	C++17 command-line application and browser interface
Team	Apurva Anand (25BCS12080), Tabish Khan (25BCS12094)
Live website	file-compresson.netlify.app
Source code	github.com/apurvaanand51/file-compressor-cpp

Contents

Abstract..... 3
1. Introduction..... 3
2. Problem Statement..... 3
3. Objectives, Scope, and Assumptions..... 3
4. Repository and Folder Structure..... 4
5. Huffman Algorithm..... 5
6. System Design and Implementation..... 8
7. HUF1 File Format..... 9
8. Data Structures and Complexity..... 10
9. User Interfaces..... 11
10. Testing and Validation..... 12
11. Build, Run, and Deploy..... 12
12. Design Decisions, Limitations, and Future Work..... 13
13. Conclusion..... 14
Project Resources..... 14

Huff: A Huffman File Compressor for Text and Binary Files

A C++17 command-line tool and a browser interface that share one custom archive format (HUF1)

Abstract

Huff is a lossless file-compression project based on Huffman coding. It provides a C++ command-line program and a browser-based interface for compressing and restoring text and binary files. Both interfaces use the same custom HUF1 format, so an archive produced by one can be restored by the other. The compressor counts the frequency of every possible byte, constructs a binary Huffman tree, and replaces each input byte with its variable-length prefix code. The archive stores the original byte count and the frequency table so that the decompressor can rebuild the same tree. The implementations validate archive structure, decoded frequencies, padding bits, and trailing data. Browser processing happens locally; selected files are not uploaded to a server.

Keywords — *Huffman coding; lossless compression; prefix codes; priority queue; bit-level I/O; file-format design; C++17; browser application.*

1. Introduction

File compression reduces the number of bits needed to represent data. Huffman coding is a lossless compression algorithm: frequently occurring symbols receive shorter codes, while less frequent symbols receive longer codes. The generated codes are prefix-free, which means that no symbol's code is the beginning of another symbol's code. This property makes the bit stream unambiguous to decode.

This project applies Huffman coding to bytes rather than only to text characters. A byte-oriented design supports plain text, images, documents, and other binary files using the same algorithm. The project demonstrates frequency analysis, priority queues, binary trees, bit-level input/output, file-format design, error handling, and automated testing.

2. Problem Statement

Create a practical program that can encode arbitrary files into a self-describing archive and restore them exactly. The solution should:

- Preserve every input byte in a compress/decompress round trip.
- Support all 256 possible byte values.
- Provide both a command-line tool and an accessible browser interface.
- Use one documented file format across both interfaces.
- Reject malformed or incomplete compressed data.
- Keep the source file intact and avoid uploading browser-selected files.

3. Objectives, Scope, and Assumptions

3.1 Objectives

1. Implement Huffman tree construction with a min-priority queue.
2. Encode symbols as variable-length prefix codes and pack bits into bytes.
3. Store enough metadata to rebuild the decoding tree.
4. Provide CLI compression/decompression with optional output paths.
5. Provide an in-browser interface for file selection, compression, and restoration.

6. Test ordinary input, edge cases, arbitrary bytes, and malformed archives.

3.2 Scope and Assumptions

The alphabet is the set of byte values `0` through `255`; characters are not interpreted as text or decoded using a particular character encoding. The archive uses unsigned 64-bit values for original size and symbol frequencies.

> ***Compression is lossless, but it does not guarantee a smaller output.*** *The header and frequency table add overhead, especially for small files and data with nearly uniform symbol frequencies.*

The C++ compressor requires a seekable input stream. It reads the stream once to count frequencies, then rewinds it for encoding. The browser reads the selected file into memory, so very large files may be better handled with the C++ program.

4. Repository and Folder Structure

The repository separates the reusable compression library, command-line program, browser UI, and tests. The core algorithm does not depend on filenames or a particular user interface.

4.1 Folder Layout

```
file-compressor-cpp/
|-- CMakeLists.txt        Build targets, compiler settings, test registration
|-- README.md             User-facing overview, setup, CLI, screenshots
|-- project-readme.md     Detailed report
|-- study-guide.md        Algorithm notes and short viva questions
|-- sample.txt            Example input file
|-- include/
|   `-- huffman.hpp       Public stream-based codec declarations
|-- src/
|   |-- huffman.cpp       Huffman algorithm and HUF1 serialization
|   `-- main.cpp          CLI arguments and filesystem operations
|-- tests/
|   `-- test_huffman.cpp  Round-trip and malformed-input tests
|-- web/
|   |-- index.html        Browser UI structure, styles, accessibility
|   `-- app.js            Browser-side HUF1 encoder and decoder
|-- assets/               Interface and result screenshots (img1-img3.png)
`-- build/                Generated CMake files, executables, libraries
```

4.2 Main Files

- `include/huffman.hpp` declares `huffman::compress` and `huffman::decompress`. Each accepts an input stream and an output stream, so a caller controls the file paths while the codec handles the data.
- `src/huffman.cpp` implements frequency counting, tree construction, code generation, bit-level I/O, and HUF1 validation.
- `src/main.cpp` parses CLI arguments, chooses default output names, opens files in binary mode, protects against using the same input and output file, and reports errors.
- `tests/test_huffman.cpp` tests the public codec API using string streams, without depending on temporary files.

- `web/index.html` and `web/app.js` implement the browser interface and a JavaScript version of the same archive format.
- `CMakeLists.txt` configures the C++17 library and executables, enables strict compiler warnings, and registers tests with CTest.
- `build/` is generated build output, not source. `.gitignore` excludes it and also excludes generated `.huf` and `.restored` files.

5. Huffman Algorithm

5.1 Tree Construction

Let *f(s)* be the number of times byte symbol *s* appears in the input. The compressor first calculates *f(s)* for every possible byte. It then builds a binary tree from symbols with nonzero frequency:

1. Create a leaf for every occurring symbol and insert it into a min-priority queue, ordered by frequency.
2. Remove the two nodes with the lowest frequencies.
3. Create a parent node whose frequency is the sum of those two nodes.
4. Insert the parent into the queue.
5. Repeat until one node remains; this node is the root.
6. Assign `0` to a left edge and `1` to a right edge. A symbol's code is the path from the root to its leaf.

Leaves represent symbols, while internal nodes combine subtrees. No leaf is an ancestor of another leaf, so no symbol code is a prefix of another. During decoding, the reader follows each bit from the root until it reaches a leaf, emits that byte, and begins the next symbol at the root.

5.2 Deterministic Tree Construction

Two nodes may have the same frequency. Both implementations resolve ties consistently by comparing frequency first, then the smallest byte value in each subtree, and then node creation order. This deterministic ordering is important: the archive stores frequencies rather than explicit tree structure, so the encoder and decoder must rebuild the same tree.

5.3 Worked Example

For the input `AAAABBC`, the frequencies are `A: 4`, `B: 2`, and `C: 1`.

1. The initial queue contains `C` (weight 1), `B` (weight 2), and `A` (weight 4).
2. The two least-frequent nodes, `C` and `B`, are combined into a parent of weight 3.
3. That parent is combined with `A`, producing a root of weight 7.
4. With the deterministic symbol ordering, the resulting codes are `A: 1`, `C: 00`, and `B: 01`.
5. Encoding the input in order produces `1111010100`: ten data bits packed into two bytes, with zero padding in the remaining low-order bits.

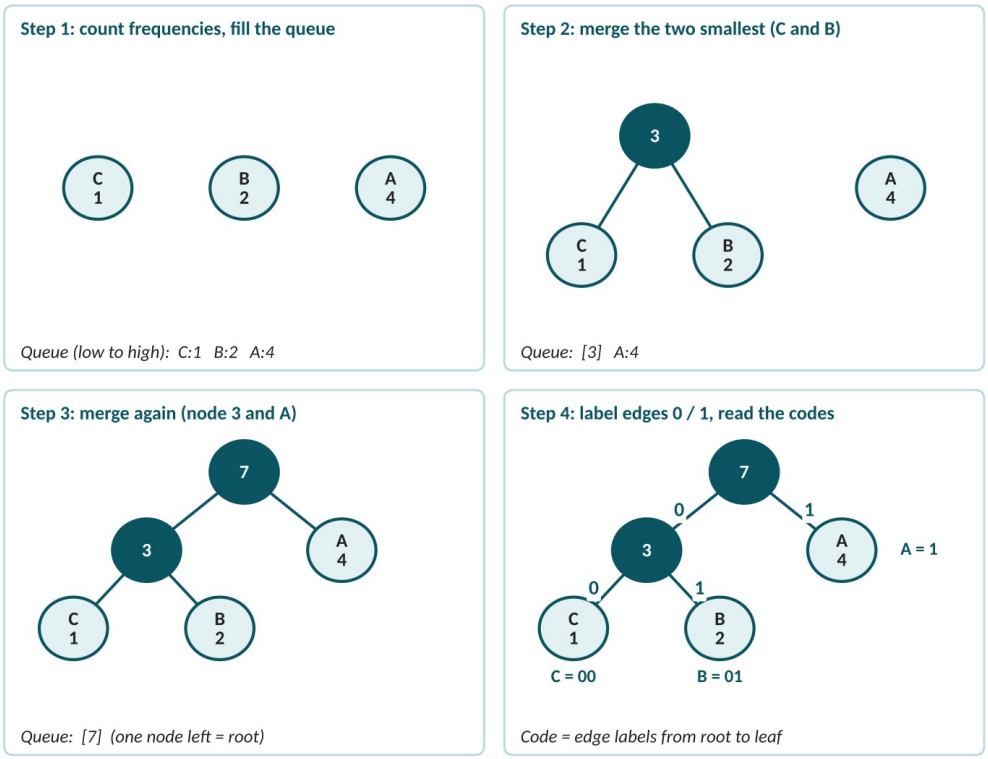


Figure 1: Building the Huffman tree for AAAABBC, one merge at a time.

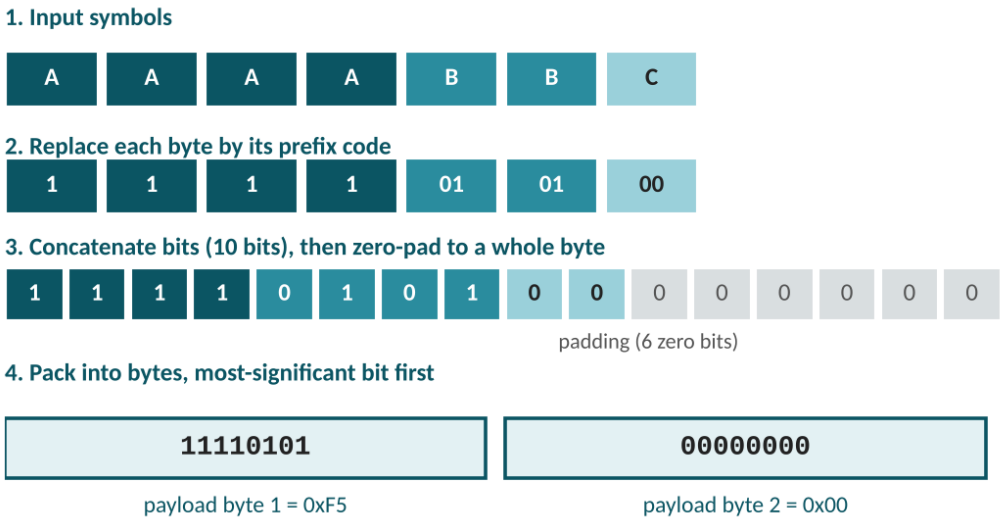


Figure 2: Turning codes into bytes: concatenate, zero-pad, pack MSB first.

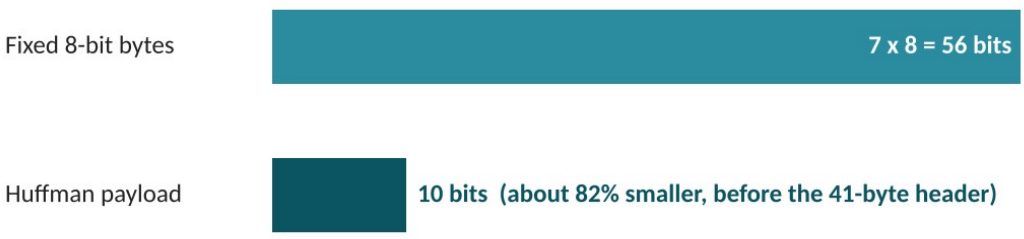


Figure 3: Payload size for AAAABBC with fixed 8-bit bytes versus Huffman codes.

The exact bit patterns are determined by the tree ordering. The archive does not store these bit strings: it stores the frequencies required to rebuild the same tree. For this tiny input the archive is 14 + 9 × 3 = 41 header bytes plus 2 payload bytes, i.e. 43 bytes for a 7-byte file, a concrete illustration of the metadata overhead discussed in Section 9.3.

5.4 Decoding Walkthrough

Decoding reverses the process. The decoder rebuilds the same tree from the stored frequencies, then reads the payload one bit at a time. Because the codes are prefix-free, the first leaf reached always ends a symbol, so no separators between codes are needed. Figure 4 decodes the payload from the worked example.

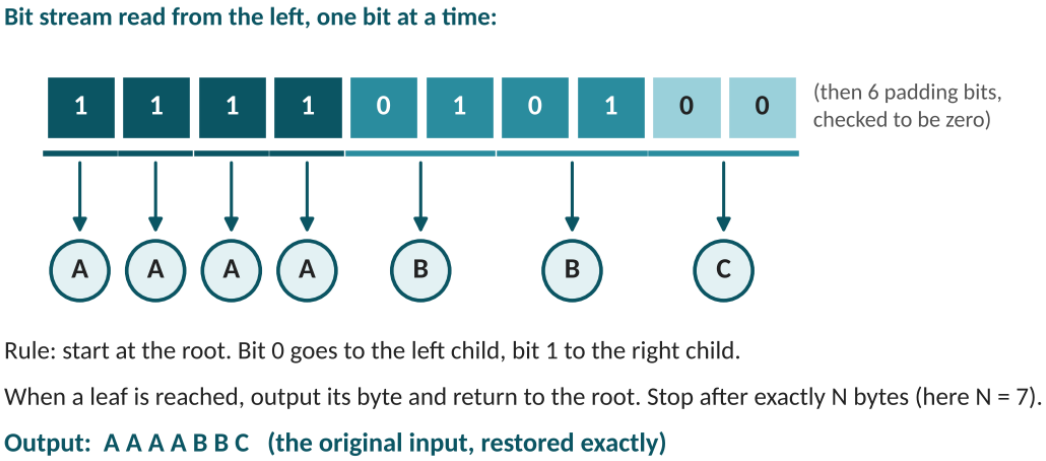


Figure 4: Decoding 1111010100 by walking the tree from the root.

5.5 Pseudocode

```
compress(input):
    count[0..255] = 0
    for each byte b in input:
        count[b] += 1
    write magic, original size, symbol count, nonzero frequencies
    tree  = build_tree(count)
    codes = paths_from_root(tree)
    for each byte b in input:
        write codes[b] to the bit stream
    zero-pad the last output byte if needed

decompress(archive):
    read and validate magic, original size, and frequencies
    tree = build_tree(frequencies)
    repeat original size times:
        follow payload bits from root to a leaf
        emit the leaf's byte
    validate decoded frequencies, padding, and end of archive
```

6. System Design and Implementation

6.1 Architecture

The project has four logical components:

Table 1: Logical components

Component	Responsibility
C++ codec library	Stream-based compression/decompression independent of paths and UI.
Command-line application	Arguments, file opening, output naming, and user-facing errors.
Browser application	File picker/drop zone, processing states, and download link.
Tests	Round-trip correctness and rejection of malformed input.

The C++ codec is exposed through streams so the CLI can handle files without loading the entire source into memory. The browser uses a separate JavaScript codec that reads and writes the same HUF1 format. CMake builds the C++ library, CLI, and test executable.

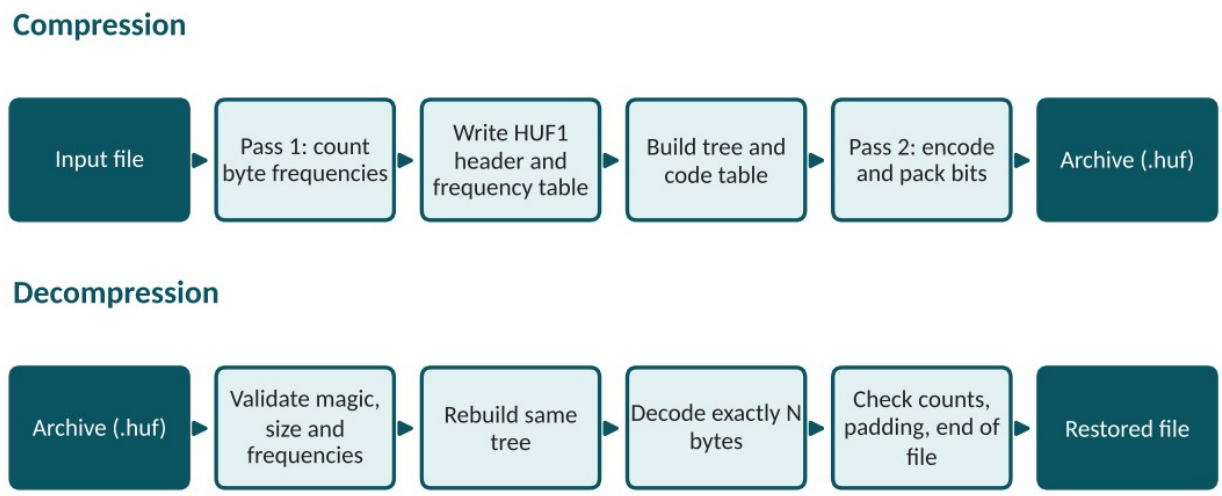


Figure 5: Compression and decompression pipelines.

6.2 Tree and Code-Table Implementation

Each C++ tree node stores its frequency, the smallest symbol in its subtree, the symbol value (internal nodes use a sentinel), left and right children, and a creation-order value. Leaf nodes represent bytes found in the input; internal nodes represent the combination of two subtrees. Nodes are owned by `unique_ptr` objects in a vector, while the priority queue stores non-owning pointers to those stable objects.

The C++ `std::priority_queue` comparator makes the lowest-frequency node the next node removed. After building the tree, a recursive traversal creates a 256-entry code table. It appends `0` while descending left and `1` while descending right, then removes the last bit when returning to visit a sibling.

6.3 Compression Flow

1. Record the input stream's current position and fail if it is not seekable.
2. Read chunks of up to 8192 bytes. For each byte, increment its frequency and the total size, checking both against the 64-bit format limits.

3. Clear the end-of-file state and rewind the input stream.
4. Write the HUF1 identifier, original size, symbol count, and nonzero frequency entries in ascending byte-symbol order.
5. Build the tree and code table.
6. Read the file again, look up each byte's code, and send its bits to the bit writer.
7. Check for input/output errors and flush any partially filled byte with zero padding.

Two passes allow the frequency table to appear before the encoded payload without keeping the whole source file in memory. The cost is that the stream must support seeking.

6.4 Bit-Level Output and Input

The encoder writes code bits, not textual characters such as `'0'` and `'1'`. `BitWriter` shifts each bit into an 8-bit accumulator. When eight bits have been collected, it writes one byte and resets the accumulator. If the final byte is only partly filled, it shifts used bits toward the most-significant side. The unused low-order positions then contain zero padding.

`BitReader` reads each payload byte from its most-significant bit toward its least-significant bit. Each bit selects a left or right tree edge. Reaching a leaf emits that byte and resets traversal to the root. After producing the declared original size, the decoder validates unused padding and rejects unexpected trailing bytes.

6.5 Decompression Flow and Validation

1. Read exactly four bytes and require the magic identifier `HUF1`.
2. Read the original-size and symbol-count fields.
3. Reject impossible combinations, such as a zero original size with a nonzero symbol count.
4. Read the frequency entries. Reject truncated entries, zero frequencies, duplicate symbols, integer overflow, and a frequency sum that differs from the declared original size.
5. Rebuild the tree using the deterministic ordering.
6. Decode exactly the declared number of bytes, tracking the count produced for each symbol.
7. Require the decoded counts to match the frequency table; check zero padding and end-of-file.

An empty input is represented by a 14-byte header with no symbols or payload. For a single-symbol input, the symbol and its count fully describe the output, so no encoded payload bits are written.

6.6 Command-Line Behavior

The CLI accepts `compress` and `decompress` commands, an input path, and an optional output path. By default, compression appends `.huf` to the input name, and decompression appends `.restored`. The application normalizes paths and checks filesystem equivalence where possible to avoid using the same file as input and output. Files are opened in binary mode to prevent platform-specific newline conversions.

Invalid usage prints the help text and returns a usage error status. Processing errors are caught at the application boundary and printed to standard error. The reusable codec throws descriptive runtime errors for read, write, and archive-validation failures.

7. HUF1 File Format

7.1 Layout

All multibyte unsigned integers are stored in little-endian byte order.

Table 2: HUF1 archive fields

Field	Size	Description
Magic identifier	4 bytes	ASCII `HUF1`.
Original size	8 bytes	Number of bytes in the original file.
Symbol count	2 bytes	Number of distinct byte values, from 0 to 256.
Frequency entries	9 bytes each	One symbol byte followed by its 8-byte frequency.
Encoded payload	Remaining bytes	Huffman bits packed most-significant-bit first.

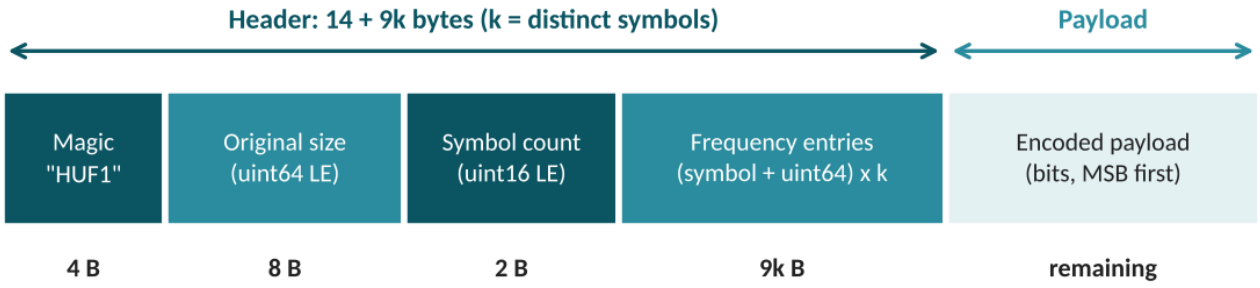


Figure 6: Byte layout of a HUF1 archive.

The fixed header occupies 14 bytes. Each distinct input byte adds 9 bytes of frequency metadata. For k distinct symbols, header size is therefore `14 + 9k` bytes. Payload size depends on the code length assigned to each input symbol and is rounded up to a whole byte.

7.2 Header Validation

The decoder reads fields in this order:

1. Magic: exactly four bytes; must equal `HUF1`.
2. Original size: unsigned 64-bit count of output bytes.
3. Symbol count: unsigned 16-bit count from 0 through 256.
4. Frequency entries: one byte identifying each symbol, followed by its unsigned 64-bit frequency.
5. Payload: packed Huffman bits, if any.

Each listed frequency must be nonzero, symbols must not be repeated, and the sum of frequencies must equal the original size.

> ***No checksum.*** *The format does not contain a checksum or cryptographic hash, so structural validation does not authenticate data or detect every corruption that could still produce a structurally valid archive.*

8. Data Structures and Complexity

8.1 Data Structures

- **Frequency array:** 256 counters, one for each possible byte.
- **Min-priority queue:** repeatedly selects the two least-frequent nodes.
- **Binary tree:** represents the prefix-code structure.

- **Code table:** maps each occurring byte to its bit path.
- **Bit writer/reader:** convert between individual code bits and bytes.

8.2 Time Complexity

Let *n* be the number of input bytes, *k* the number of distinct byte values ($0 \le k \le 256$), and *L* the maximum Huffman code length.

- Counting frequencies takes `O(n)` time.
- C++ tree construction with a min-priority queue takes `O(k log k)`.
- Encoding and decoding take `O(n * L)` time in the general case. With this fixed 256-symbol alphabet, *L* is bounded by 255 bits, so processing is linear in file size with respect to *n*, though code lengths can vary.

8.3 Memory Use

Tree, frequency, and code-table memory is bounded by the byte alphabet (`O(k)` tree/code storage plus 256 counters). C++ compression uses an 8192-byte read buffer and stores tree/code metadata, rather than the whole input file. C++ decompression writes output incrementally.

The browser implementation loads the selected and resulting file data into memory. Its tree builder sorts an array of at most 256 nodes during each merge; the C++ implementation uses a priority queue.

9. User Interfaces

9.1 Command Line

```
file-compressor compress   <input-file> [output-file]
file-compressor decompress <input-file> [output-file]
file-compressor --help
```

Compression defaults to `<input-file>.huf`; decompression defaults to `<input-file>.restored`. An optional output path overrides the default. Example on Windows:

```
.\build\file-compressor.exe compress .\sample.txt
.\build\file-compressor.exe decompress .\sample.txt.huf
```

9.2 Browser Application

The web page offers Compress and Decompress tabs. Users can choose a file with the browser picker or drag it into the drop area. The selected filename and size are displayed before processing. After the operation, the page reports the result and provides a download link. The decompress picker filters for `.huf` files, while the decoder independently checks the file contents.

The JavaScript implementation reads the input using the browser File API and processes bytes in `Uint8Array` buffers. It uses `BigInt` for frequencies and 64-bit fields. Results are wrapped in a `Blob`; a temporary object URL powers the download link and is revoked when the result is cleared or replaced. Before allocating a decompression output buffer, the browser checks that the declared size is within its safe array-size range.

Errors are displayed in the page. The interface distinguishes an actual reduction from a file that grows because of metadata overhead, and it handles empty-file output explicitly. Files are processed locally in the browser, not sent to the Netlify site or another server. The live interface is available at file-compresson.netlify.app.

9.3 Interpreting Compression Results

For original size *N* and compressed size *C*, the page calculates a size reduction when `C <= N` as:

```
reduction (%) = 100 * (1 - C / N)
```

When `C > N`, it reports the percentage growth instead. Compression is not always beneficial: *C* includes the header and frequency table as well as the encoded payload. Small files and high-entropy files can therefore produce larger archives.

10. Testing and Validation

10.1 Round-Trip Tests

The C++ test executable checks byte-for-byte round trips for:

- Empty input.
- A 4096-byte single-symbol input.
- Ordinary text.
- All 256 byte values, including non-printable bytes.
- Varied binary data.

10.2 Malformed-Input Tests

Table 3: Malformed-archive test cases

Test case	What it checks
Invalid magic	Non-`HUF1` data is rejected.
Truncated payload	Missing bits needed for decoding cause an error.
Trailing payload byte	Data after a valid code stream is not accepted.
Nonzero padding	Unused low-order payload bits must be zero.
Single-symbol trailing data	The special case also requires a clean end of archive.

Round-trip tests compare the restored bytes to the exact original bytes, so they verify losslessness rather than only checking that decompression finishes. Tests are registered with CTest under `huffman-round-trip`.

10.3 Running the Tests

```
cmake -S . -B build
cmake --build build
ctest --test-dir build --output-on-failure
```

11. Build, Run, and Deploy

11.1 Build and Run

The C++ project requires CMake 3.16 or newer and a C++17-compatible compiler. It has no third-party library dependencies. Configure, build, and test from the project directory:

```
cmake -S . -B build
cmake --build build
ctest --test-dir build --output-on-failure
```

To open the browser application locally, open `web/index.html` directly or serve the repository directory using a local web server and navigate to `/web/index.html`.

11.2 Netlify Deployment

The browser UI is a static website. Connect the GitHub repository in Netlify, leave the build command empty, and set the publish directory to `web`. No server-side codec or build step is required; compression and decompression run in the visitor's browser.

12. Design Decisions, Limitations, and Future Work

12.1 Design Decisions

- **Byte-oriented symbols:** one codec handles text and arbitrary binary formats without character-encoding assumptions.
- **Frequency table instead of stored tree:** the header stores the data needed to rebuild the tree. Stable tie-breaking is consequently required for cross-implementation compatibility.
- **Two-pass C++ compression:** lets the compressor write the frequency metadata before the payload without keeping the complete source in memory.
- **Stream-based C++ API:** separates algorithm behavior from file paths and the command-line interface.
- **Shared archive format:** enables C++ and browser implementations to exchange compressed files.
- **Standard-library C++ and built-in web APIs:** avoid external runtime dependencies.

12.2 Limitations

- Header and frequency metadata can outweigh payload savings on small or high-entropy files.
- The C++ compressor requires a seekable input stream; the current header design cannot be emitted from a non-seekable source in one pass.
- The browser holds input and output buffers in memory. Large files can require substantial memory, and practical limits vary by browser.
- HUF1 has no checksum. Structural validation does not authenticate data or detect every possible corruption.
- The project is intended as a learning implementation, not as a replacement for general-purpose formats such as ZIP, gzip, or zstd.

12.3 Future Enhancements

1. Define a versioned archive format with checksums for stronger corruption detection.
2. Add progress reporting and cancellation to longer browser operations.
3. Explore chunked browser processing to reduce peak memory use.
4. Benchmark text, binary, small, and already-compressed files, including metadata overhead.
5. Add cross-language tests that compress in C++ and decompress in JavaScript, and vice versa.

13. Conclusion

Huff demonstrates the full lifecycle of a practical lossless compression system: frequency analysis, priority-queue-based tree construction, prefix code generation, bit-level encoding, a documented binary format, archive validation, and user-facing tools. A shared format connects the C++ and browser implementations, while round-trip and malformed-input tests check important correctness properties.

Project Resources

- **GitHub repository:** https://github.com/apurvaanand51/file-compressor-cpp
- **Live website:** https://file-compresson.netlify.app/
- **Additional study guide:** `study-guide.md`